



ASSESSMENT COVER SHEET

(To be adapted as needed on module/school basis)

Please complete this cover sheet for each assessment submission. For group assessments, please ensure each group member completes and submits a completed individual cover sheet.

STUDENT (REGISTRATION) NUMBER: 40841063

*Please ensure that you have removed your name from your assessment – don't forget to check both the header and the footer. Please **do** include your student number though.*

MODULE NUMBER: SET09802 2025-6 TR3 001

MODULE TITLE: Software Development 1

ASSESSMENT TITLE: Dice Poker Coursework 2

NAME OF MODULE LEADER: Divya Achhodawala

DATE OF SUBMISSION: Friday August 21st

I declare, except where explicit acknowledgement is made of contributions from other sources*, that this assessment is the result of my own work and has not been submitted for any module or degree programmed at Edinburgh Napier University or any other institution. This is in accordance with Edinburgh Napier University's [Academic Integrity Regulations](#).

***IMPORTANT:** Contributions from other sources' may include incorrectly cited quotations or Artificial Intelligence (AI) tools, such as ChatGPT or Bard (details of which can be found in the [Guidelines for Students on AI & Writing Assistant Tools](#)). **Please declare here whether you have used such tools in this assessment submission, and to what extent:**

(Select one of the two options below)

☐ No, I have not used AI for this submission.

☒ Yes, I have used AI for this submission.

If you have used AI tools in completing this submission, please briefly describe in approximately 100 words how you have used these tools:

I have used AI tools to check grammar for my paragraphs, with careful practices not to change any original meaning or context of my text. As English is not my first language, there are some minor grammar mistakes in my report. To fix these, AI tools Grammarly and Quill bot are used in order to improve my writing and make it more academic.

Table of Contents

ASSESSMENT COVER SHEET	1
Dice Game Program.....	5
Program introduction	5
Class Diagram	5
Class Descriptor (Main Features Only)	6
Main Class	6
Dice Game Class.....	6
Betting Program Class	6
Dice Rolling Class	7
Reward Conditions.....	7
Round Recorder	8
Scoreboard Class.....	8
Result Board Class.....	8
Extensibility Code.....	8
Separation of Methods	8
Method Reusability	9
Reward Condition Class	9
Number of rounds	9
Sides of die.....	11
Number of dice	11
Advanced Features.....	13
Variable bet amount	13
High Score Table.....	14
Additional Features	15

Currency Changer	15
Dice Fighter	18
Updated Class Diagram	20

Dice Game Program

Program introduction

The dice game program includes the functions where the user can enter usernames, make bets, and play the game. The game allows users to five times, although the game will end, when there is no more bank balance. The class diagram for the entire program can be seen below.

Class Diagram

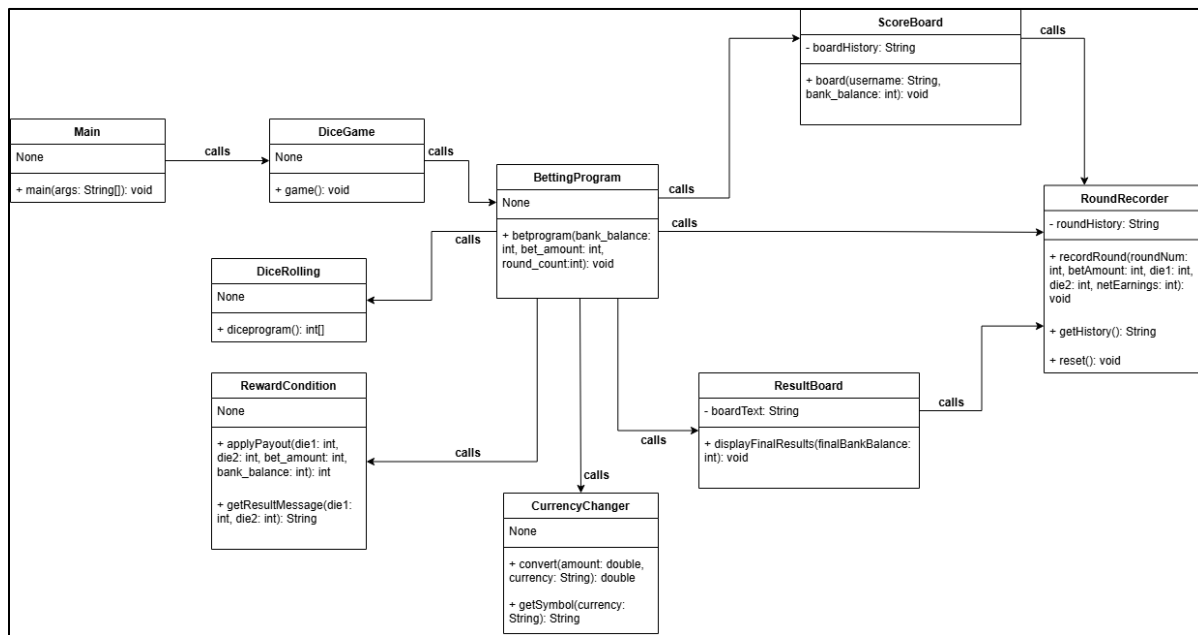


Figure 1 Class Diagram

Each class in the diagram has their own purpose and every class calls other classes for code reusability. The purpose of each class is –

- Main → starts the program
- Dice Game → controls the game
- Betting Program → handles betting programs
- Dice Rolling → rolls the dice
- Reward Condition → calculates rewards based on winning condition
- Round Recorder → record each round history after each loop
- Score Board → displays score board

- Result Board → displays player's results
- Currency Changer → additional class that allow users to see their final balance in other currencies

Class Descriptor (Main Features Only)

This section of the report will mention in detail how each class works and how they are related or make use of other classes for reusability and clean coding.

Main Class

This is the base class of the whole program and where the program will be run. The main class only contains main method, where Dice Game class is called inside it.

Dice Game Class

The dice game class is the median class between the main class and the betting program class. The dice game class calls betting program with the parameters set in the class.

Betting Program Class

Betting class is the foundation class for the whole program. Most of the codes for each function are either written in the class or called from other classes. The functions of the betting program class include –

- Ask username of the players
- Shows username and starting bank balance
- Ask users to enter a bet amount (Until valid amount is entered)
- Deduct bet amount from the bank account
- Count rounds
- Calls dice rolling class to roll dice
- Access results from the returned list
- Set new bank balance using reward condition class
- Record each round with each loop (loop until bank account is 0 or round count ≤ 5)
- After the loop ends, check bank account to see ending condition and show players
- Calls result board class to show the game history

- Calls score board class to show player's ranking among dummy users

Dice Rolling Class

The dice rolling class is specifically written to roll the dice and is called by the betting program. This improves the code reusability, as the class can be called again and again, whenever we need to roll the dice. The program includes dice program method with the random rolling function.

Reward Conditions

The class checks the winning conditions of the dice game. The code is separated into another class to improve readability and cleanliness. The class is called by the betting program to see the winning conditions of the player and if their bet has doubled, tripled, or lost the round. The condition checking is called within each loop of the betting program until the game is finished. The class consists of two methods, apply payout and get result message, to give users both String and calculated output for bank balance.

```
1 // HLUM HTET AUNG (40841063)
2
3 package part2DiceGame;
4
5 public class RewardCondition {
6     // Called from BettingProgram to check reward condition
7     public static int applyPayout(int die1, int die2, int bet_amount, int bank_balance) {
8         if (Math.abs(die1 - die2) == 1) {
9             bank_balance += (bet_amount * 2);
10        } else if (die1 == die2) {
11            bank_balance += (bet_amount * 3);
12        }
13
14        return bank_balance;
15    }
16
17    public static String getResultMessage(int die1, int die2) {
18        if (Math.abs(die1 - die2) == 1) {
19            return "Congrats! Your bet has doubled.";
20        } else if (die1 == die2) {
21            return "Congrats! Your bet has tripled.";
22        } else {
23            return "Sorry, you did not win this time.";
24        }
25    }
26 }
```

Figure 2 Rewarding conditions

Round Recorder

The round recorder is a class that records the results of each round and stores them as history. The class is called by betting program, scoreboard class, and result board class. The class contains recording round method to record rounds, a get method to return history, and reset method which reset the recording.

Scoreboard Class

The class is for displaying user's scoreboard. It is called a betting program and consists of board method. This method uses username parameters entered in betting program to set player's name, and board method uses username and bank balance to place a ranking of user among dummy players. This method mainly uses simple Arrays.sort() to tackle the ranking problem.

Result Board Class

The class is for displaying user's results and works almost the same as high score board. However, this board contains only one method that displays the result of the betting history of the user, and the class is called by the betting program.

Extensibility Code

The dice game program is extensible, and the program variables are very easy to change. The program uses professional practices and other means to ensure that functions can be added, modified, or deleted without rewriting most of the code.

Separation of Methods

The dice game program separates methods for better code reusability and clean coding. For example, codes from specific classes can be added in future class, rather than writing the same code twice. The same goes for clean coding, where if there is an error or logical mistake in the code, it is easier to go to the specific class and fix the bug than search through the complex single code base file.

Method Reusability

Methods in the program, such as `DiceRolling.diceprogram()`, are reusable as they are a standalone function. For example, if there is a need to roll the dice in another class, this method can be reused in another class for quick and easy dice rolling. And the same form of dice rolling result is returned in the class that is called the method. This is good for reusing the specific classes in the future so that time isn't wasted on the same code. Reusable classes and methods in the dice game program –

- `DiceRolling.diceprogram()`
- `ResultBoard.recordRound()`
- `ScoreBoard.board()`
- `RoundRecorder.recordRound()`

Apart from these mainly reusables, other methods from some classes, such as `RewardCondition.applyPayout()`, are also reusable in order to apply rewards to bank balance in the future games.

Reward Condition Class

This class defines all the reward conditions of the whole program. This means that if there should be the need to add or modify reward payouts or winning conditions, we can only modify this class and not touch the whole code. This reduces debugging problems, where if the function is fixed in the combined code base, it can cause errors and additional conflicts.

Number of rounds

In here the number of rounds is passed into the betting program as a parameter so that when there is a need to modify the number of rounds, users can simply modify the variable parameter in the wrapper class dice game and pass it into betting program.

```

1 // HLUM HTET AUNG (40841063)
2
3 package part2DiceGame;
4
5 public class DiceGame {
6
7     public static void game() { // Final game object and called from main
8         final int bank_balance = 6;
9         int round_count = 1;
10        int bet_amount = 0;
11        final int maxRound = 5;
12
13        BettingProgram.betprogram(bank_balance, bet_amount, round_count, maxRound);
14
15    }
16
17 }
18 }
19

```

Figure 3 Dice Game max round variable

```

Main.java  DiceGame.java  BettingProgram.java  RewardCondition.java  RoundRecorder.java  ScoreBoard.java  ResultBoard.java
54
55    round_count++;
56    // Storing returning results from dice program in local variables
57    int[] dicerresults = DiceRolling.diceprogram();
58    int die1 = dicerresults[0];
59    int die2 = dicerresults[1];
60    JOptionPane.showMessageDialog(null, "Die 1: " + die1 + "\nDie 2: " + die2);
61    System.out.println("Die 1 rolled " + die1 + " and Die 2 rolled " + die2);
62
63    // Assigning return value from an object into a variable
64    int newBalance = RewardCondition.applyPayout(die1, die2, bet_amount, bank_balance);
65    int netEarnings = newBalance - bank_balance;
66    bank_balance = newBalance;
67    String rewardmsg = RewardCondition.getResultMessage(die1, die2) + "\nBank balance: £" + bank_balance;
68
69    JOptionPane.showMessageDialog(null, rewardmsg);
70
71    // Record results to display on final board
72    RoundRecorder.recordRound(round_count - 1, bet_amount, die1, die2, netEarnings);
73
74    } while (bank_balance > 0 && round_count <= maxRound);
75
76    // Game ending condition
77    if (bank_balance == 0) {
78        JOptionPane.showMessageDialog(null, "Game Over: You ran out of money!");
79        System.out.println("Game Over: You ran out of money!");
80    } else if (round_count > 5) {
81        JOptionPane.showMessageDialog(null, "Game Over: You completed 5 bets!");
82        System.out.println("Game Over: You completed " + maxRound + " bets!");
83    }
84
85    // Output final result board to both Console and JOptionPane
86    ResultBoard.displayFinalResults(bank_balance);
87
88    // ADVANCED FEATURE 2: displaying score board
89    ScoreBoard.board(username, bank_balance);
90
91    }
92 }
93

```

Figure 4 Betting program use of max round parameter

Sides of die

In the `diceRolling.diceprogram()`, the number of the die faces are not hard coded and is assigned to a variable. What problem does this solve? This gets rid of the hard coding, where if there is a need to change die face, it is possible to modify a single variable rather than changing 6 in every line of code.

```
1 // HLUM HTET AUNG (40841063)
2
3 package part2DiceGame;
4
5 import java.util.Random;
6
7 public class DiceRolling {
8     public static int[] diceprogram() { // called from BettingProgram for rolling dice
9         int die1result;
10        int die2result;
11        final int dieFace = 6;
12
13        Random random = new Random();
14        die1result = random.nextInt(dieFace) + 1;
15        die2result = random.nextInt(dieFace) + 1;
16
17        return new int[]{die1result, die2result};
18    }
19 }
20
```

Figure 5 Die face variable extensibility

Here the face of die can be changed by simply modifying the die face variable and not each integer in each line.

Number of dice

The number of dice in the first draft was hardcoded. This can limit the ability for the user to easily modify the number of dice.

```

Main.java  DiceGame.java  BettingProg...  RewardCondi...  RoundRecorde...  DiceRolling.java
42         JOptionPane.showMessageDialog(null, "The b
43         + bank_bal
44     } else {
45         break;
46     }
47 }
48
49 // Deduct bet from bank balance
50 bank_balance -= bet_amount;
51 JOptionPane.showMessageDialog(null, "Bet accepted!
52 "\nRemaining bank balance: " + bank_balance);
53 System.out.println("Bet amount is " + bet_amount +
54
55 round_count++;
56 // Storing returning results from dice program in
57 int[] dicerresults = DiceRolling.diceprogram();
58 int die1 = dicerresults[0];
59 int die2 = dicerresults[1];
60 JOptionPane.showMessageDialog(null, "Die 1: " + di
61 System.out.println("Die 1 rolled " + die1 + " and
62
1 // HLUM HTET AUNG (40841063)
2
3 package part2DiceGame;
4
5 import java.util.Random;
6
7 public class DiceRolling {
8     public static int[] diceprogram() { // called fr
9         int die1result;
10        int die2result;
11        final int dieFace = 6;
12
13        Random random = new Random();
14        die1result = random.nextInt(dieFace) + 1;
15        die2result = random.nextInt(dieFace) + 1;
16
17        return new int[]{die1result, die2result};
18    }
19 }
20

```

Figure 6 Number of dice

This issue is resolved by setting a dice variable and using loop so that there is only one list variable.

```

DiceRolling.java
1 HLUM HTET AUNG (40841063)
2
3 package part2DiceGame;
4
5 import java.util.Random;
6
7 public class DiceRolling {
8     public static int[] diceprogram() { // called from
9         final int diceCount = 2;
10        int[] diceArray = new int[diceCount];
11        final int dieFace = 6;
12        int dieresult;
13
14        Random random = new Random();
15        for (int i=0; i < diceCount; i++) {
16            dieresult = random.nextInt(dieFace) + 1;
17            diceArray[i] = dieresult;
18        }
19
20        return diceArray;
21    }
22 }
23

```

Figure 7 Dice rolling with a dice count variable

By setting die number in die rolling class, the number of dice can be set and accessed from the betting program using die 1, die 2, or die 3 variables easily. Thus, the number of dice can be changed easily.

Advanced Features

The advanced features on the dice game program were tackled using the user input variable for a bet between £1 and £4, and the score board class for the user score to display by ranking based on the bank balance among the other dummy players.

Variable bet amount

The variable bet amount is the first advanced feature and in the dice game program, a separate variable for the user to enter bet amount is created, along with the constraints that limit the user from entering invalid numbers.

```
// Prompt until a valid bet is entered
while (true) {
    String input = JOptionPane.showInputDialog("Enter bet amount (£1 to £4):");

    // Cancel button exit
    if (input == null) {
        System.exit(0);
    }

    // ADVANCED FEATURE 1: variable bet amount, Exception check on valid integer
    try {
        bet_amount = Integer.parseInt(input);
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(null, "Please enter a valid number.");
        continue; // Restart the input loop
    }

    // If condition for valid bank balance
    if (bet_amount < 1 || bet_amount > 4) {
        JOptionPane.showMessageDialog(null, "Bet amount must be between £1 and £4.");
    } else if (bet_amount > bank_balance) {
        JOptionPane.showMessageDialog(null, "The bet amount is higher than your bank balance. Bank balance: "
            + bank_balance);
    } else {
        break;
    }
}
```

Figure 8 Variable bet amount

The code block above is the bet amount input function, where the users are required to enter a valid number between the specified bet amount and the system exit to exit the system, if the user chooses to cancel the game.

High Score Table

The high score table is the second advanced feature that shows the ranking of the user based on their bank balance after the game had finished. This was completed using high score class, which includes a list to store static dummy users and the variable username and final bank balance of the user. First, after the game is over, the username and bank balance is added into the list, and the list is sorted using `Arrays.sort()`. After the array is sorted, ranking numbers are added in order, and the results are shown in a table. The user will be ranked according to their bank balance and the higher their balance is compared to the dummy players, the higher their position in the table will be.

```
// Call this ONCE at game over
public static void board(String username, int bank_balance) {
    // 1. Array of [Name, Balance] including 10 dummy players + active player
    String[][] players = {
        {"Ian", "30"}, {"Charlie", "25"}, {"Emma", "18"},
        {"Grace", "15"}, {"Alice", "12"}, {"Jack", "10"},
        {"Frank", "8"}, {"Bob", "4"}, {"Hannah", "2"}, {"David", "0"},
        {username, String.valueOf(bank_balance)}
    };

    // 2. Sort by balance directly from Highest to Lowest
    Arrays.sort(players, (a, b) -> Integer.parseInt(b[1]) - Integer.parseInt(a[1]));

    // 3. Build text string
    String output = "\n\nGAME SUMMARY\n"
        + "Player: " + username + "\n"
        + "-----\n"
        + RoundRecorder.getHistory() + "\n"
        + "===== TOP RANKINGS =====\n";

    for (int i = 0; i < players.length; i++) {
        String name = players[i][0];
        String bal = players[i][1];
        String marker = name.equals(username) ? " <-- YOU" : "";

        output += "#" + (i + 1) + " " + name + " : £" + bal + marker + "\n";
    }

    output += "=====";

    // 4. Output to Console and JOptionPane
    System.out.println(output);
    JOptionPane.showMessageDialog(null, output, "Leaderboard", JOptionPane.INFORMATION_MESSAGE);

    // Reset history for next game run
    RoundRecorder.reset();
}
```

Figure 9 Code block for high score table

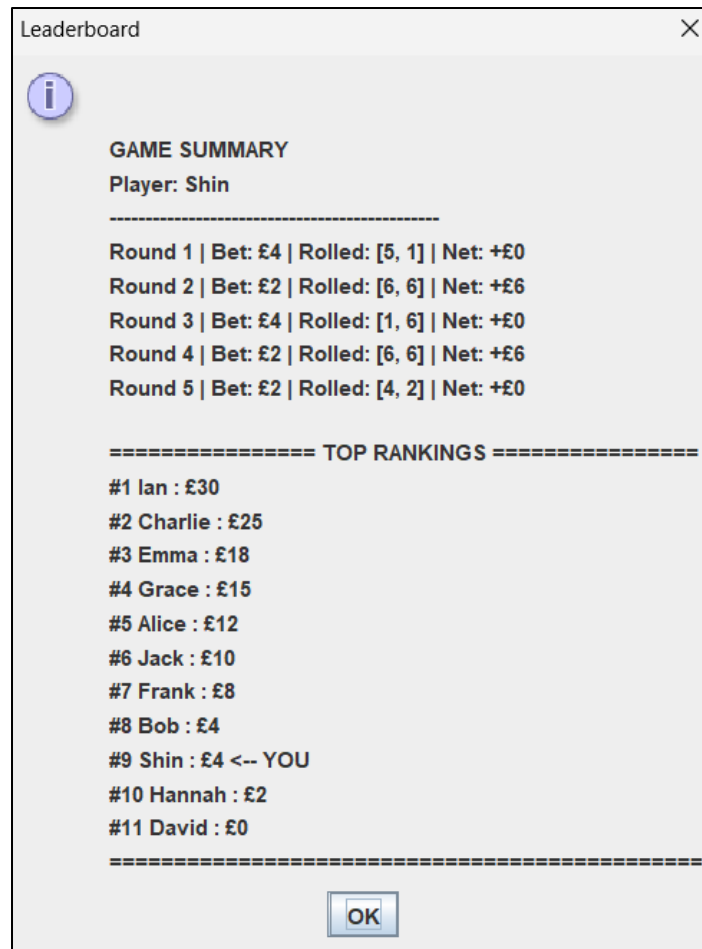


Figure 10 Score board result

Additional Features

This section is the additional features for the program to be better prepared for deployment. The add-on features include – currency changer, dice fight (player vs computer).

Currency Changer

For the base game dice poker, the program will include an additional feature, where the user will be allowed to see their final remaining bank balance in other currencies. The currency will include £ (the base currency), \$ (USD), mmk (Myanmar Kyat), and jpy (Japanese Yen). The implementation steps will be explained below.

```

1 package part2DiceGame;
2
3 public class CurrencyChanger {
4     public static double convert(double amount, String currency) {
5
6         switch (currency) {
7             case "USD":
8                 return amount * 1.35;
9
10            case "EUR":
11                return amount * 1.16;
12
13            case "MMK":
14                return amount * 2800;
15
16            case "GBP":
17                return amount;
18
19            default:
20                return amount;
21        }
22    }
23
24    public static String getSymbol(String currency) {
25        switch (currency) {
26            case "USD": return "$";
27            case "EUR": return "€";
28            case "MMK": return "MMK";
29            case "GBP": return "£";
30            default: return "£";
31        }
32    }
33 }
34

```

Figure 11 Currency Changer Class

This is the currency changer class with the additional currency calculations and the symbol method for displaying final bank balance in each symbol accordingly.

```

// See final bank balance in other currencies
int choice = JOptionPane.showConfirmDialog(null, "Do you want to see your remaining bank balance in other currency?",
    "Currency Selection", JOptionPane.YES_NO_OPTION);
if (choice == JOptionPane.YES_OPTION) {
    // Bank Balance Currency Converter
    String[] currencies = {"GBP", "USD", "EUR", "MMK"};

    String currency = (String) JOptionPane.showInputDialog(
        null,
        "Choose your currency:",
        "Currency Selection",
        JOptionPane.QUESTION_MESSAGE,
        null,
        currencies,
        currencies[0]
    );

    if (currency == null) {
        System.exit(0);
    }

    String symbol = CurrencyChanger.getSymbol(currency);

    JOptionPane.showMessageDialog(null, "Your bank balance in " + symbol + ": " +
        CurrencyChanger.convert(bank_balance, currency));
}

```

Figure 12 Betting Program currency converter code block

The function is implemented at the end of the class, after the game is over to let users choose if they want to see their bank balance in other currencies. If the choice is yes, users will need to select the currency, and the Currency Changer class will do the calculations and display the result. Otherwise, the function will be skipped and result table and scoreboard will be displayed.

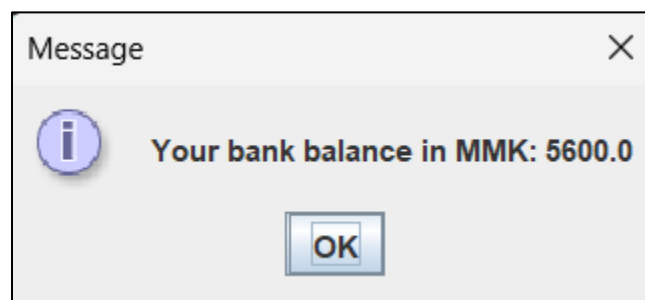


Figure 13 Currency Changer Test Result

In this test result the amount is shown in Myanmar currency (Kyat). After the game ended, the bank balance was £2, and it is calculated to be MMK 5600, with Currency Changer class.

Dice Fighter

This is an additional game where users can play dice fighting with the computer. The core mechanic of the game is both player and computer roll the dice using dice rolling function, written in dice poker game. The numbers are then compared and whoever has the higher number wins. This mini game is written to demonstrate reusability.

```
1 // HLUM HTET AUNG (40841063)
2
3 package part2DiceGame;
4
5 import javax.swing.JOptionPane;
6
7 public class DiceFighter {
8     public static void dicefight(int roundCount, int maxRound) {
9         int userwins = 0;
10        int computerwins = 0;
11        int user_total, computer_total;
12
13        int choice = JOptionPane.showConfirmDialog(null, "Do you want to play dice fighter?",
14            "Select Yes or No", JOptionPane.YES_NO_OPTION);
15
16        if (choice == JOptionPane.YES_OPTION) {
17
18            do {
19                int[] userResults = DiceRolling.diceprogram();
20                int[] computerResults = DiceRolling.diceprogram();
21                user_total = userResults[0] + userResults[1];
22                computer_total = computerResults[0] + computerResults[1];
23
24                JOptionPane.showMessageDialog(null, "Round " + roundCount + "\n===== \n"
25                    + "The dice roll total for user: "
26                    + user_total + "\n" + "The dice roll total for computer: " + computer_total);
27
28                System.out.println("The dice roll total for user: " + user_total);
29                System.out.println("The dice roll total for computer: " + computer_total);
30
31                if (user_total > computer_total) {
32                    userwins += 1;
33                } else {
34                    computerwins += 1;
35                }
36
37                roundCount += 1;
38            } while (roundCount < maxRound);
39        } else {
40            DiceGame.game();
41        }
42
43        if (userwins > computerwins) {
44            JOptionPane.showMessageDialog(null, "After " + roundCount + " rounds, User won the game.");
45            System.out.println("After " + roundCount + " rounds, User won the game.");
46        } else if (computerwins > userwins) {
47            JOptionPane.showMessageDialog(null, "After " + roundCount + " rounds, Computer won the game.");
48            System.out.println("After " + roundCount + " rounds, Computer won the game.");
49        } else if (computerwins == userwins) {
50            JOptionPane.showMessageDialog(null, "After " + roundCount + " rounds, The game is draw.");
51            System.out.println("After " + roundCount + " rounds, The game is draw.");
52        }
53
54
55    }
56
57 }
58
59 }
```

Figure 14 Dice Fighter game code

Every code was written in a single class and like dice poker, the user is expected to play a specific number of rounds. The game is called from dice game text, where initially had only dice poker

call. Now after adding dice fighter, there is a conditional statement in dice game where it ask users if they want to play dice fighter or dice poker.

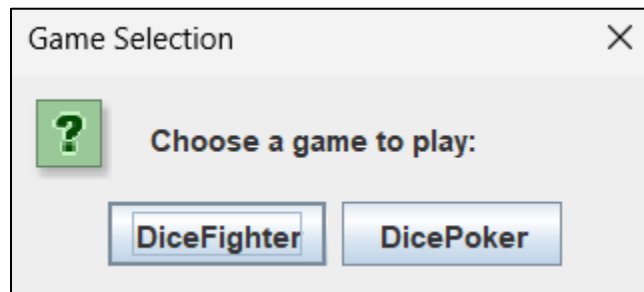


Figure 15 Game selection

```
1 // HLUM HTET AUNG (40841063)
2
3 package part2DiceGame;
4
5 import javax.swing.JOptionPane;
6
7 public class DiceGame {
8
9     public static void game() { // Final game object and called from main
10         final int bank_balance = 6;
11         int round_count = 1;
12         int bet_amount = 0;
13         final int maxRound = 5;
14
15         String choice = askUser();
16
17         if (choice.equals("DiceFighter")) {
18             DiceFighter.dicefight(round_count, maxRound);
19         }
20         else if (choice.equals("DicePoker")) {
21             BettingProgram.betprogram(bank_balance, bet_amount, round_count, maxRound);
22         }
23     }
24
25     public static String askUser() {
26
27         String[] options = {"DiceFighter", "DicePoker"};
28
29         int choice = JOptionPane.showOptionDialog(null, "Choose a game to play:", "Game Selection", JOptionPane.DEFAULT_OPTION,
30             JOptionPane.QUESTION_MESSAGE, null, options, options[0]
31         );
32
33         if (choice == 0) {
34             return "DiceFighter";
35         }
36         else if (choice == 1) {
37             return "DicePoker";
38         }
39
40         return ""; // User closed the window
41     }
42
43 }
44 }
45 }
```

Figure 16 Dice game class modification

This additional feature is added to ensure that code blocks are reusable.

Updated Class Diagram

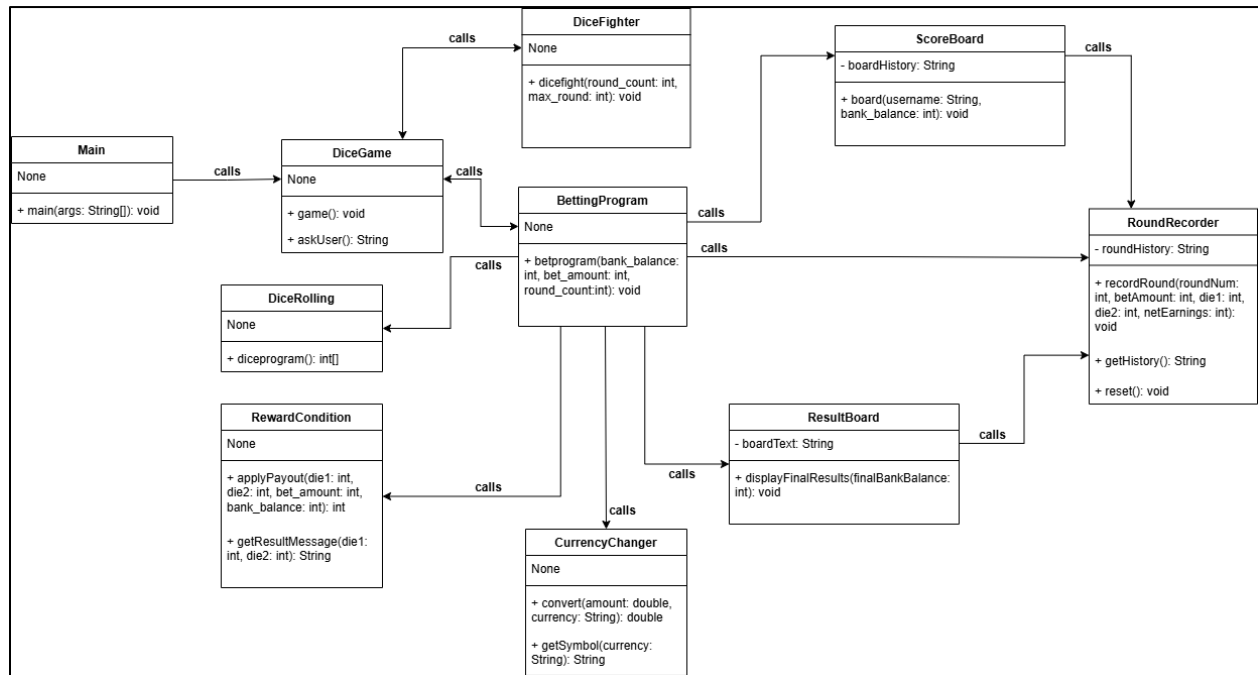


Figure 17 Updated Class Diagram